

Numerical Methods for Solving Nonlinear Equations and Systems of Nonlinear Equations

Serhet Gökdemir
21058024

Advisor: Prof. Dr. Hale Gonca Köçken

Istanbul, 2026

List of Figures

5.1	Convergence comparison of single-variable methods	40
5.2	Convergence comparison of system methods	41

List of Tables

3.1	Bisection method for $f(x) = x^3 - x - 2$	16
3.2	Secant method for $f(x) = x^3 - x - 2$	18
3.3	Newton's method for $f(x) = x^3 - x - 2$	19
3.4	Damped Newton method for $f(x) = x^3 - x - 2$	21
3.5	Brent's method for $f(x) = x^3 - x - 2$	23
3.6	General comparison of the methods for the illustrative example	23
4.1	Newton method for the nonlinear system	31
4.2	Broyden method for the nonlinear system	33
5.1	Comparison of single-variable methods	40
5.2	Comparison of system methods	41

Symbols

$f(x)$	Scalar nonlinear function
$F(\mathbf{x})$	Vector-valued nonlinear function
x^*	Exact root or solution
x_k	Approximation at iteration k
$\mathbf{x}_k$	Vector approximation at iteration k
e_k	Error at iteration k
$J_F(\mathbf{x})$	Jacobian matrix of F
$\mathbf{s}_k$	Iteration step vector
α_k	Damping or step-size parameter
ε	Convergence tolerance
$\ \mathbf{x}\ $	Euclidean norm of vector $\mathbf{x}$
B_k	Approximate Jacobian matrix in Broyden's method
p	Order of convergence
C	Convergence constant
$\mathbf{0}$	Zero vector
$\mathbb{R}^n$	n -dimensional real vector space
$\phi(x)$	Residual or objective function
Δ_k	Trust-region radius
$g(x)$	Fixed-point iteration function
L	Contraction constant

Abbreviations

NumPy Numerical Python Library

Pandas Python Data Analysis Library

Preface

This thesis has been prepared as part of the undergraduate studies in Mathematical Engineering at Yıldız Technical University.

The main motivation behind this work is the author's interest in numerical methods and optimization techniques. During the study, an effort has been made to understand the fundamental principles of solving nonlinear equations and to explore their practical implementations.

The author would like to express sincere gratitude to the advisor, Prof. Dr. Hale Gonce Kken, for her guidance, support, and valuable insights throughout the preparation of this thesis.

Abstract

This thesis investigates numerical methods for solving nonlinear equations and systems of nonlinear equations. Such problems arise frequently in various areas of science, engineering, optimization, and computational mathematics, and in many practical cases their exact analytical solutions cannot be obtained. For this reason, numerical approaches play an important role in approximating their solutions.

The study focuses on both single nonlinear equations and systems of nonlinear equations. For single-variable problems, the bisection method, secant method, Newton's method, damped Newton method, and Brent's method are examined. For nonlinear systems, multivariable Newton and Broyden methods are studied. The theoretical foundations, convergence properties, advantages, limitations, and failure cases of these methods are discussed.

In addition to the theoretical analysis, the selected numerical methods are implemented in Python using a modular computational structure. Numerical experiments are performed in order to compare the methods in terms of convergence behavior, stability, and computational efficiency. Convergence plots, residual analyses, and iteration histories are used to evaluate the performance of the algorithms.

The thesis also investigates the relationship between nonlinear equation solving and optimization techniques. In particular, connections between Newton-type methods, damping strategies, line-search approaches, trust-region methods, and residual minimization are discussed from an optimization perspective.

Overall, the study combines theoretical numerical analysis with practical computational implementation and provides a structured examination of numerical methods for nonlinear problems.

Contents

List of Figures	i
List of Tables	ii
Symbols	iii
Abbreviations	iv
Preface	v
Abstract	vi
Contents	vii
1 Introduction	1
1.1 Motivation Behind Numerical Methods	1
1.2 Nonlinear Equations and Systems	1
1.3 Objectives of the Thesis	2
1.4 Structure of the Thesis	3
2 Mathematical Background	4
2.1 Functions and Roots	4
2.2 Fixed-Point Formulation	4
2.3 Convergence of Iterative Methods	4
2.4 Order of Convergence	5
2.5 Norms and Vector Functions	5
2.6 Jacobian Matrix	5
2.7 Stopping Criteria	6
3 Single Nonlinear Equations	7
3.1 Bisection Method	7
3.2 Secant Method	8
3.3 Newton's Method	10
3.4 Damped Newton Method	11
3.5 Brent's Method	12
3.6 Illustrative Example	15
3.6.1 Bisection Method	15
3.6.2 Secant Method	16
3.6.3 Newton's Method	18

3.6.4	Damped Newton Method	19
3.6.5	Brent's Method	21
3.6.6	Comparison for the Example	22
3.6.7	Failure Cases	23
3.7	Comparison of Methods	25
4	Systems of Nonlinear Equations	26
4.1	Multivariable Newton Method	26
4.2	Convergence Analysis	27
4.3	Broyden's Method	28
4.4	Illustrative Example	29
4.4.1	Newton Method for Systems	30
4.4.2	Broyden's Method for Systems	32
4.4.3	Comparison for the Example	33
4.4.4	Failure Cases	33
4.5	Comparison of Methods	35
5	Implementation and Numerical Experiments	36
5.1	Implementation Details	36
5.1.1	Code Organization and Reproducibility	36
5.2	Test Problems	38
5.3	Results and Comparisons	39
5.3.1	Single-Variable Experiments	39
5.3.2	System Experiments	40
5.4	Discussion	41
6	Connection with Optimization	43
6.1	Residual Minimization Perspective	43
6.2	Newton's Method and Optimization	44
6.3	Damped Newton Method and Line Search	45
6.4	Trust-Region Perspective	46
6.5	Numerical Illustration of Damping Effects	47
7	Conclusion	49
7.1	Summary of Findings	49
7.2	Limitations	49
7.3	Future Work	50
	Bibliography	51
	A Resume	52

1. Introduction

1.1 Motivation Behind Numerical Methods

There are two basic approaches one may apply while trying to solve any mathematical problem: solving analytically or numerically. The process of solving analytically refers to calculating the exact solution of the given problem. It is possible to calculate the solution of many classical mathematical problems analytically and thus to derive the exact solution.

Nevertheless, there are many problems for which the analytical solution either does not exist at all or its calculation is excessively expensive. For instance, nonlinear and systems of nonlinear equations often cannot be solved analytically; thus, other methods have to be used.

Unlike the analytic approach, numeric methods allow deriving an approximate solution with a required precision. These methods utilize iterative algorithms that make it possible to gradually improve an approximation up until the required precision is obtained.

Different numerical methods have their own advantages and disadvantages. These advantages and disadvantages can be their convergence rates, computational costs and stability. So, selecting an appropriate method for a given problem has an important role. [1, 2].

1.2 Nonlinear Equations and Systems

A nonlinear equation is an equation of the form

$$f(x) = 0, \tag{1.1}$$

where f is a nonlinear function. In contrast to linear equations, nonlinear equations may exhibit multiple roots, no real roots, or complex solution structures, making their analysis and solution more challenging [1, 2].

A simple example of a nonlinear equation is

$$f(x) = x^3 - x - 2 = 0 \tag{1.2}$$

for which no closed-form analytical solution exists in a simple form. Such equations commonly arise in various scientific and engineering applications, including physics, optimization, and computational modeling.

In many practical cases, it is not possible to determine the exact solution of a nonlinear equation analytically. Even when an analytical solution exists, it may be too

complicated or computationally expensive to evaluate. For this reason, numerical methods are employed to approximate the roots of nonlinear equations with a desired level of accuracy.

The problem can be extended to systems of nonlinear equations. A nonlinear system can be written in vector form as

$$F(\mathbf{x}) = \mathbf{0}, \quad (1.3)$$

where

$$F : \mathbb{R}^n \rightarrow \mathbb{R}^n \quad (1.4)$$

is a vector-valued nonlinear function.

For example, consider the system

$$\begin{cases} x^2 + y^2 - 4 = 0, \\ x - y - 1 = 0. \end{cases} \quad (1.5)$$

Such systems arise naturally in applications where multiple variables interact in a nonlinear manner.

Solving nonlinear systems is generally more complex than solving single nonlinear equations. In particular, the solution methods often require the use of the Jacobian matrix, defined as

$$J_F(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}. \quad (1.6)$$

which generalizes the concept of the derivative to higher dimensions [3]. In this thesis, both single nonlinear equations and systems of nonlinear equations are considered. The focus is on numerical methods that iteratively approximate the solution and their behavior in terms of convergence, computational cost, and robustness.

1.3 Objectives of the Thesis

The main objective of this thesis is to study numerical methods for solving nonlinear equations and systems of nonlinear equations. These problems arise frequently in various fields of science and engineering, and in many cases, their exact solutions cannot be obtained analytically. As a result, numerical approaches play a fundamental role in approximating their solutions [1, 2].

In this context, the thesis focuses on two main problem classes. First, numerical methods for solving single nonlinear equations of the form $f(x) = 0$ are investigated. Classical methods such as bisection, secant, and Newton-type methods are examined in terms of their underlying principles and convergence behavior.

Second, the study extends to systems of nonlinear equations of the form $F(\mathbf{x}) = \mathbf{0}$. These systems are generally more complex and require more advanced techniques, particularly those involving the Jacobian matrix. Methods for solving such systems are analyzed within a similar theoretical framework.

In addition to the theoretical analysis, the selected numerical methods are implemented using a computational tool. The implementations are used to evaluate the practical performance of the methods through numerical experiments. In particular, the

methods are compared in terms of convergence behavior, computational efficiency, and robustness.

Overall, the aim of this thesis is to provide a clear and structured examination of numerical methods for nonlinear problems, combining theoretical foundations with computational analysis.

1.4 Structure of the Thesis

This thesis consists of six main chapters.

Chapter 1 introduces the motivation behind numerical methods and discusses nonlinear equations and systems of nonlinear equations. The objectives and overall scope of the thesis are also presented in this chapter.

Chapter 2 provides the mathematical background required for the numerical methods studied throughout the thesis. Fundamental concepts such as fixed-point formulation, convergence analysis, order of convergence, norms, Jacobian matrices, and stopping criteria are introduced.

Chapter 3 focuses on numerical methods for solving single nonlinear equations. Classical methods including the bisection method, secant method, Newton's method, damped Newton method, and Brent's method are explained together with their convergence properties, illustrative examples, and failure cases.

Chapter 4 extends the discussion to systems of nonlinear equations. Multivariable Newton's method and Broyden's method are examined theoretically and computationally. Illustrative examples and convergence comparisons are also included.

Chapter 5 presents the implementation details and numerical experiments. The Python implementations of the numerical methods are discussed together with the repository structure, testing process, reproducibility considerations, and experimental results obtained from benchmark problems.

Finally, Chapter 6 investigates the relationship between nonlinear equation solving and optimization techniques. Connections between Newton-type methods, line-search strategies, trust-region approaches, and residual minimization are discussed from an optimization perspective.

2. Mathematical Background

2.1 Functions and Roots

In numerical analysis, the problem of solving a nonlinear equation is commonly expressed as finding a root of a function. A root of a function $f(x)$ is a value $x^* \in \mathbb{R}$ such that

$$f(x^*) = 0. \quad (2.1)$$

Depending on the structure of the function, there may exist a single root, multiple roots, or no real roots. In many practical applications, the function f is continuous, and root-finding methods often rely on this property [1, 2].

2.2 Fixed-Point Formulation

Many numerical methods are based on transforming the root-finding problem into a fixed-point problem. Instead of solving $f(x) = 0$, the equation is rewritten in the form

$$x = g(x), \quad (2.2)$$

where g is a suitably defined function.

A point x^* is called a fixed point of g if

$$x^* = g(x^*). \quad (2.3)$$

If g satisfies certain conditions, iterative methods of the form

$$x_{k+1} = g(x_k) \quad (2.4)$$

can be used to approximate the solution [1].

2.3 Convergence of Iterative Methods

An iterative method generates a sequence $\{x_k\}$ that is expected to approach the true solution x^* . The method is said to converge if

$$\lim_{k \rightarrow \infty} x_k = x^*. \quad (2.5)$$

The convergence of an iterative method depends on several factors, including the choice of the initial guess and the properties of the function.

A common condition for convergence in fixed-point iteration is that the function g is a contraction, meaning that there exists a constant $L < 1$ such that

$$|g(x) - g(y)| \leq L|x - y| \quad (2.6)$$

for all x, y in a neighborhood of the fixed point $[1, 4]$.

2.4 Order of Convergence

The speed at which an iterative method converges is described by its order of convergence. Let $e_k = |x_k - x^*|$ denote the error at iteration k . The method is said to have order p if there exists a constant $C > 0$ such that

$$e_{k+1} \approx Ce_k^p \quad (2.7)$$

for sufficiently large k .

For example:

- $p = 1$: linear convergence
- $p = 2$: quadratic convergence

Higher-order methods generally converge faster, but they may require more computational effort per iteration [1].

2.5 Norms and Vector Functions

For systems of nonlinear equations, it is necessary to measure the size of vectors. A commonly used norm is the Euclidean norm, defined as

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}. \quad (2.8)$$

Norms are used to quantify errors and convergence in higher-dimensional problems [2].

2.6 Jacobian Matrix

For a system of nonlinear equations $F(\mathbf{x}) = \mathbf{0}$, the Jacobian matrix plays a central role in many numerical methods.

The Jacobian matrix is defined as

$$J_F(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}. \quad (2.9)$$

It generalizes the concept of the derivative to multivariable functions and is essential in methods such as Newton's method for systems [3].

2.7 Stopping Criteria

In practical computations, iterative methods cannot run indefinitely. Therefore, a stopping criterion is required.

Common stopping criteria include:

- $|x_{k+1} - x_k| < \varepsilon$
- $|f(x_k)| < \varepsilon$
- maximum number of iterations reached

where ε is a predefined tolerance [1, 4].

3. Single Nonlinear Equations

3.1 Bisection Method

The bisection method is one of the simplest and most reliable numerical techniques for solving nonlinear equations of the form $f(x) = 0$. It is based on the Intermediate Value Theorem, which states that if a continuous function f changes sign over an interval $[a, b]$, then there exists at least one root in that interval [1, 2].

More precisely, if

$$f(a) \cdot f(b) < 0, \quad (3.1)$$

then there exists a point $c \in (a, b)$ such that

$$f(c) = 0. \quad (3.2)$$

The method starts with such an interval $[a, b]$ and repeatedly bisects it. At each iteration, the midpoint is computed as

$$c_k = \frac{a_k + b_k}{2}. \quad (3.3)$$

The value of the function at the midpoint, $f(c_k)$, is then evaluated. Depending on the sign of $f(c_k)$, the interval is updated as follows:

- If $f(a_k) \cdot f(c_k) < 0$, then the root lies in $[a_k, c_k]$, so set $b_{k+1} = c_k$.
- Otherwise, the root lies in $[c_k, b_k]$, so set $a_{k+1} = c_k$.

This process is repeated until a stopping criterion is satisfied, such as the interval length becoming sufficiently small or the function value approaching zero.

One of the main advantages of the bisection method is its guaranteed convergence. As long as the function is continuous and the initial interval satisfies the sign change condition, the method will always converge to a root [1].

However, the method has a relatively slow rate of convergence. The error decreases linearly, and the interval length is halved at each iteration. More precisely, after k iterations, the error satisfies

$$|x_k - x^*| \leq \frac{b_0 - a_0}{2^k}. \quad (3.4)$$

This linear convergence makes the method less efficient compared to methods such as Newton's method or the secant method. Nevertheless, due to its robustness and simplicity, the bisection method is often used as a baseline or as a component in more advanced hybrid methods such as Brent's method [2, 4].

Implementation Example. The following Python function presents a simplified implementation of the bisection algorithm used throughout the numerical experiments.

```

1 def bisection(f, a, b, tol=1e-8, max_iter=100):
2
3     fa = f(a)
4     fb = f(b)
5
6     if fa * fb >= 0:
7         raise ValueError("Invalid bracketing interval.")
8
9     history = []
10
11    for i in range(max_iter):
12
13        c = (a + b) / 2
14        fc = f(c)
15
16        history.append({
17            "iteration": i,
18            "a": a,
19            "b": b,
20            "c": c,
21            "fc": fc
22        })
23
24        if abs(fc) <= tol:
25            return c, history
26
27        if fa * fc < 0:
28            b = c
29            fb = fc
30        else:
31            a = c
32            fa = fc
33
34    return c, history

```

Listing 3.1: Python implementation of the bisection method

3.2 Secant Method

The secant method is an iterative numerical technique used to solve nonlinear equations of the form $f(x) = 0$. It can be seen as a derivative-free alternative to Newton's method, as it approximates the derivative using finite differences instead of requiring an explicit analytical expression [1, 2].

Given two initial approximations x_0 and x_1 , the method constructs a secant line through the points $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$. The next approximation is obtained by finding the root of this secant line:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}. \quad (3.5)$$

This formula eliminates the need for computing derivatives, making the method useful when the derivative of the function is difficult or expensive to evaluate.

Unlike the bisection method, the secant method does not require the initial guesses to bracket a root. However, this also means that convergence is not guaranteed. The method may fail to converge if the initial guesses are not chosen appropriately or if the function behaves poorly in the neighborhood of the root [3].

In terms of convergence speed, the secant method is generally faster than the bisection method. It has a superlinear rate of convergence, with order approximately equal to

$$p \approx 1.618, \quad (3.6)$$

which is known as the golden ratio. Although this is slower than the quadratic convergence of Newton's method, it often provides a good balance between efficiency and computational cost [1].

One limitation of the secant method is that it requires two initial values and may encounter numerical instability if $f(x_k) \approx f(x_{k-1})$, which can lead to division by very small numbers. Despite these limitations, the method is widely used in practice due to its simplicity and effectiveness [4].

Implementation Example. The following Python function shows the main computational structure of the secant method. The implementation keeps the two most recent approximations, computes the next iterate using the secant formula, and stops when either the residual or the step size becomes sufficiently small.

```

1 def secant(f, x0, x1, tol=1e-8, max_iter=100):
2
3     f0 = f(x0)
4     f1 = f(x1)
5     history = []
6
7     for i in range(max_iter):
8
9         denominator = f1 - f0
10
11         if abs(denominator) < max(10.0 * tol, 1e-15):
12             raise ValueError("Secant denominator is too small.")
13
14         x_next = x1 - f1 * (x1 - x0) / denominator
15         error = abs(x_next - x1)
16
17         history.append({
18             "iteration": i,
19             "x_prev": x0,
20             "x_curr": x1,
21             "x_next": x_next,
22             "error": error
23         })
24
25         f_next = f(x_next)
26
27         if abs(f_next) <= tol or error <= tol:
28             return x_next, history
29
30         x0, x1 = x1, x_next
31         f0, f1 = f1, f_next
32
33     return x1, history

```

3.3 Newton's Method

Newton's method, also known as the Newton–Raphson method, is one of the most widely used techniques for solving nonlinear equations of the form $f(x) = 0$. Unlike the bisection and secant methods, Newton's method makes explicit use of the derivative of the function to achieve faster convergence [1, 2, 3].

Starting from an initial guess x_0 , the method generates a sequence $\{x_k\}$ using the iterative formula

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (3.7)$$

Geometrically, this update rule corresponds to approximating the function by its tangent line at x_k and taking the root of this linear approximation as the next iterate.

One of the main advantages of Newton's method is its rapid convergence. Under suitable conditions, such as when the initial guess is sufficiently close to the true root and $f'(x^*) \neq 0$, the method exhibits quadratic convergence. This means that the number of correct digits approximately doubles at each iteration [1].

However, the performance of Newton's method strongly depends on the choice of the initial guess. If the initial value is far from the root, the method may converge slowly, diverge, or converge to a different root. In addition, the method requires the computation of the derivative $f'(x)$, which may be difficult or computationally expensive for some functions [3].

Another limitation arises when the derivative is close to zero, which can lead to large updates and numerical instability. Despite these challenges, Newton's method remains a fundamental tool in numerical analysis due to its efficiency and strong theoretical properties [5].

Implementation Example. The following Python function presents the main implementation structure of Newton's method. The derivative is evaluated at each iteration, and a safeguard is included in order to avoid division by a very small derivative.

```

1 def newton(f, df, x0, tol=1e-8, max_iter=100):
2
3     x = x0
4     history = []
5
6     for i in range(max_iter):
7
8         fx = f(x)
9         dfx = df(x)
10
11         if abs(dfx) < max(10.0 * tol, 1e-15):
12             raise ValueError("Derivative is too small.")
13
14         step = fx / dfx
15         x_next = x - step
16         error = abs(step)

```

```

17
18     history.append({
19         "iteration": i,
20         "x": x,
21         "fx": fx,
22         "dfx": dfx,
23         "step": step,
24         "error": error
25     })
26
27     if error <= tol or abs(f(x_next)) <= tol:
28         return x_next, history
29
30     x = x_next
31
32     return x, history

```

Listing 3.3: Python implementation of Newton’s method

3.4 Damped Newton Method

The damped Newton method is a modification of the classical Newton’s method designed to improve its stability and convergence behavior. While the standard Newton method can converge very rapidly near the solution, it may fail or diverge if the initial guess is not sufficiently close to the root. The damped version addresses this issue by introducing a step size parameter [3, 5].

The iteration formula is given by

$$x_{k+1} = x_k - \alpha_k \frac{f(x_k)}{f'(x_k)}, \quad (3.8)$$

where $\alpha_k \in (0, 1]$ is a damping parameter. When $\alpha_k = 1$, the method reduces to the classical Newton’s method.

The main idea is to control the size of the update step in order to ensure more stable convergence. In practice, α_k is often chosen using a line search strategy, which selects a step size that sufficiently reduces the function value or a related merit function [3].

One common approach is to require that the function value decreases at each iteration, that is,

$$|f(x_{k+1})| < |f(x_k)|. \quad (3.9)$$

This condition helps prevent large and potentially unstable updates, especially when the derivative is small or when the function is highly nonlinear.

Compared to the standard Newton method, the damped version generally converges more reliably from a wider range of initial guesses. However, this increased robustness comes at the cost of slower convergence, particularly when the method is already close to the solution.

The damped Newton method provides a useful balance between efficiency and stability, and it also establishes a direct connection with optimization techniques, where similar step size control strategies are widely used [6].

Implementation Example. The following implementation demonstrates the main structure of the damped Newton method together with a simple backtracking strategy for adaptive step-size control.

```

1 def damped_newton(
2     f, df, x0,
3     tol=1e-8,
4     alpha=1.0,
5     max_iter=100
6 ):
7
8     x = x0
9     history = []
10
11     for i in range(max_iter):
12
13         fx = f(x)
14         dfx = df(x)
15
16         if abs(dfx) < 1e-15:
17             raise ValueError("Derivative is too small.")
18
19         current_alpha = alpha
20
21         while abs(
22             f(x - current_alpha * fx / dfx)
23         ) >= abs(fx):
24             current_alpha *= 0.5
25
26         step = current_alpha * fx / dfx
27         x_next = x - step
28
29         history.append({
30             "iteration": i,
31             "x": x,
32             "alpha": current_alpha
33         })
34
35         if abs(f(x_next)) <= tol:
36             return x_next, history
37
38         x = x_next
39
40     return x, history

```

Listing 3.4: Python implementation of the damped Newton method

3.5 Brent's Method

Brent's method is a hybrid numerical algorithm designed to combine the robustness of bracketing methods with the fast convergence behavior of interpolation-based methods [2, 5].

Like the bisection method, Brent's method starts with an interval

$$[a, b]$$

satisfying

$$f(a)f(b) < 0,$$

which guarantees that at least one root exists inside the interval.

The main idea of the algorithm is to preserve this bracketing condition during the iteration process while attempting to accelerate convergence using interpolation techniques whenever possible. Instead of always computing the midpoint of the interval, the method dynamically chooses between different approximation strategies according to the behavior of the function.

Brent's method combines three different approaches:

- the bisection method,
- the secant method,
- inverse quadratic interpolation.

The bisection step provides reliability because it always preserves the bracketing interval. However, its convergence is relatively slow. The secant and interpolation steps are generally much faster, but they may occasionally produce unstable approximations. Brent's method attempts to balance these advantages and disadvantages automatically.

Suppose that three previous approximations

$$x_{k-2}, \quad x_{k-1}, \quad x_k$$

are available. If the function values are distinct, inverse quadratic interpolation can be applied. In this approach, the function values are treated as independent variables and a quadratic polynomial is constructed for the inverse relation

$$x = f^{-1}(y).$$

The next approximation is then estimated by evaluating this interpolating polynomial at

$$y = 0.$$

A commonly used form of the inverse quadratic interpolation formula is

$$\begin{aligned} x_{k+1} = & \frac{f(x_{k-1})f(x_k)}{(f(x_{k-2}) - f(x_{k-1}))(f(x_{k-2}) - f(x_k))}x_{k-2} \\ & + \frac{f(x_{k-2})f(x_k)}{(f(x_{k-1}) - f(x_{k-2}))(f(x_{k-1}) - f(x_k))}x_{k-1} \\ & + \frac{f(x_{k-2})f(x_{k-1})}{(f(x_k) - f(x_{k-2}))(f(x_k) - f(x_{k-1}))}x_k. \end{aligned}$$

This approximation often produces much faster convergence than simple interval bisection when the function behaves smoothly near the root.

If only two approximations are available, Brent's method may instead use the secant formula

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Although interpolation steps are usually efficient, they are not always reliable. The estimated point may fall outside the bracketing interval or may fail to reduce the residual sufficiently. In such situations, Brent's method automatically switches back to the safer bisection step

$$x_{k+1} = \frac{a+b}{2}.$$

The above mentioned adaptivity criterion is perhaps the most significant characteristic of the Brent algorithm. At each iteration, the algorithm considers if interpolation will enhance convergence. In case it cannot be ensured, the algorithm slows down, thereby providing stability at the cost of decreased speed.

Unlike the Newton method, which needs the derivative of the function, the Brent method requires no derivative computations. Moreover, Brent's method is usually several times faster than other bracketing techniques, such as the bisection method. For these reasons, the Brent method is employed in many scientific computing packages nowadays [4, 5].

Even though the full implementation of the Brent method is relatively more complex than that of traditional methods of finding roots of nonlinear equations, the high efficiency of the method in practice makes it one of the most important numerical methods.

Implementation Example. The following simplified implementation fragment illustrates the hybrid structure of Brent's method. The algorithm dynamically switches between interpolation and bisection steps in order to balance convergence speed and numerical stability.

```

1 for i in range(max_iter):
2
3     if abs(e) > tol and abs(fa) > abs(fb):
4
5         # interpolation step
6         s = fb / fa
7
8         if a == c:
9             # secant step
10            p = 2.0 * m * s
11            q = 1.0 - s
12
13        else:
14            # inverse quadratic interpolation
15            ...
16
17        # accept interpolation only if safe
18        if interpolation_is_safe:
19            d = p / q
20        else:
21            d = m
22
23    else:
24        # fall back to bisection
25        d = m
26
27    b = b + d
28    fb = f(b)

```


3.6 Illustrative Example

In order to make the behavior of the methods more concrete, the same nonlinear equation is used throughout this chapter:

$$f(x) = x^3 - x - 2 = 0. \quad (3.10)$$

Since

$$f(1) = -2 \quad \text{and} \quad f(2) = 4,$$

there is at least one real root in the interval $[1, 2]$. The approximate root is

$$x^* \approx 1.52138.$$

This example is suitable for all five methods discussed in this chapter. The aim here is not to give a fully detailed manual computation for every iteration, but to show a short sequence of iterates so that the convergence behavior can be observed clearly.

3.6.1 Bisection Method

For the bisection method, the initial interval is taken as

$$[1, 2].$$

Since

$$f(1) = 1^3 - 1 - 2 = -2$$

and

$$f(2) = 2^3 - 2 - 2 = 4,$$

we have

$$f(1)f(2) < 0.$$

Therefore, the interval $[1, 2]$ contains at least one root.

At the first iteration, the midpoint is

$$c_1 = \frac{1+2}{2} = 1.5.$$

Evaluating the function at this point gives

$$f(1.5) = 1.5^3 - 1.5 - 2 = -0.125.$$

Since

$$f(1.5) < 0 \quad \text{and} \quad f(2) > 0,$$

the sign change occurs on the interval

$$[1.5, 2].$$

Thus, the interval is updated as

$$[a_2, b_2] = [1.5, 2].$$

At the second iteration,

$$c_2 = \frac{1.5 + 2}{2} = 1.75.$$

Then

$$f(1.75) = 1.75^3 - 1.75 - 2 = 1.60938.$$

Since

$$f(1.5) < 0 \quad \text{and} \quad f(1.75) > 0,$$

the new interval becomes

$$[a_3, b_3] = [1.5, 1.75].$$

At the third iteration,

$$c_3 = \frac{1.5 + 1.75}{2} = 1.625.$$

Evaluating the function gives

$$f(1.625) = 1.625^3 - 1.625 - 2 = 0.66602.$$

Since

$$f(1.5) < 0 \quad \text{and} \quad f(1.625) > 0,$$

the interval is updated as

$$[a_4, b_4] = [1.5, 1.625].$$

Table 3.1 summarizes the first several iterations.

Table 3.1: Bisection method for $f(x) = x^3 - x - 2$

Iteration	a_k	b_k	Midpoint c_k	$f(c_k)$
1	1.00000	2.00000	1.50000	-0.12500
2	1.50000	2.00000	1.75000	1.60938
3	1.50000	1.75000	1.62500	0.66602
4	1.50000	1.62500	1.56250	0.25220
5	1.50000	1.56250	1.53125	0.05911

The bisection method converges steadily by preserving an interval that contains the root. However, the convergence is relatively slow because the interval length is reduced by only one half at each iteration.

3.6.2 Secant Method

For the secant method, the initial approximations are selected as

$$x_0 = 1, \quad x_1 = 2.$$

These initial values are chosen because the function changes sign between the two points:

$$f(1) = -2, \quad f(2) = 4.$$

In addition, the selected points are sufficiently separated from each other, which helps construct a meaningful secant line during the first iteration.

Unlike Newton's method, the secant method does not require explicit derivative information. Instead, the derivative is approximated using two consecutive iterates:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this approximation into Newton's iteration formula gives the secant iteration:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

At the first iteration,

$$x_0 = 1, \quad x_1 = 2,$$

with

$$f(1) = -2, \quad f(2) = 4.$$

Substituting these values into the iteration formula gives

$$x_2 = 2 - 4 \cdot \frac{2 - 1}{4 - (-2)}.$$

Thus,

$$x_2 = 2 - \frac{4}{6} = 1.33333.$$

Evaluating the function at the new point:

$$f(1.33333) = 1.33333^3 - 1.33333 - 2 \approx -0.96296.$$

At the second iteration,

$$x_1 = 2, \quad x_2 = 1.33333,$$

and

$$f(x_1) = 4, \quad f(x_2) \approx -0.96296.$$

The next approximation becomes

$$x_3 = 1.33333 - (-0.96296) \frac{1.33333 - 2}{-0.96296 - 4}.$$

After simplification,

$$x_3 \approx 1.46269.$$

Evaluating the function:

$$f(1.46269) \approx -0.33334.$$

At the third iteration,

$$x_4 = 1.46269 - (-0.33334) \frac{1.46269 - 1.33333}{-0.33334 - (-0.96296)}.$$

This gives

$$x_4 \approx 1.53117.$$

Table 3.2 summarizes the first several iterations.

The secant method converges faster than the bisection method because it uses interpolation information instead of repeatedly halving the interval. However, the method may become unstable if two consecutive function values become very close to each other.

Table 3.2: Secant method for $f(x) = x^3 - x - 2$

Iteration	x_{k-1}	x_k	x_{k+1}
1	1.00000	2.00000	1.33333
2	2.00000	1.33333	1.46269
3	1.33333	1.46269	1.53117
4	1.46269	1.53117	1.52093
5	1.53117	1.52093	1.52138

3.6.3 Newton's Method

For Newton's method, the initial approximation is selected as

$$x_0 = 1.5.$$

This value is chosen because it lies close to the expected root inside the interval

$$[1, 2].$$

Newton's method is highly sensitive to the initial approximation. Selecting a point sufficiently close to the root generally improves convergence and reduces the risk of divergence.

The nonlinear equation considered is

$$f(x) = x^3 - x - 2,$$

with derivative

$$f'(x) = 3x^2 - 1.$$

Newton's iteration formula is

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

At the first iteration,

$$x_0 = 1.5.$$

Evaluating the function and derivative:

$$f(1.5) = 1.5^3 - 1.5 - 2 = -0.125,$$

and

$$f'(1.5) = 3(1.5)^2 - 1 = 5.75.$$

Substituting into Newton's formula:

$$x_1 = 1.5 - \frac{-0.125}{5.75}.$$

Thus,

$$x_1 \approx 1.52174.$$

Evaluating the function at the new approximation:

$$f(1.52174) \approx 0.00214.$$

At the second iteration,

$$f'(1.52174) = 3(1.52174)^2 - 1 \approx 5.94707.$$

The next approximation becomes

$$x_2 = 1.52174 - \frac{0.00214}{5.94707}.$$

After simplification,

$$x_2 \approx 1.52138.$$

Evaluating the function:

$$f(1.52138) \approx 5.89 \times 10^{-7}.$$

At the third iteration,

$$x_3 = 1.52138 - \frac{5.89 \times 10^{-7}}{5.94379}.$$

This gives

$$x_3 \approx 1.52138.$$

Table 3.3 summarizes the iterations.

Table 3.3: Newton's method for $f(x) = x^3 - x - 2$

Iteration	x_k	$f(x_k)$	Error
0	1.50000	-0.12500	0.02174
1	1.52174	0.00214	0.00036
2	1.52138	5.89×10^{-7}	9.92×10^{-8}

The numerical results show that Newton's method converges very rapidly once the iteration approaches the root. The residual decreases dramatically within only a few iterations, which is consistent with the quadratic convergence behavior discussed earlier.

3.6.4 Damped Newton Method

For the damped Newton method, the same nonlinear equation

$$f(x) = x^3 - x - 2$$

is considered.

The initial approximation is selected as

$$x_0 = 2.$$

This starting point is intentionally chosen farther from the root compared to the standard Newton example. The purpose is to illustrate how damping can produce a more controlled iteration sequence when the initial approximation is less favorable.

The derivative of the function is

$$f'(x) = 3x^2 - 1.$$

The damped Newton iteration is defined by

$$x_{k+1} = x_k - \alpha \frac{f(x_k)}{f'(x_k)},$$

where

$$0 < \alpha \leq 1$$

is the damping parameter.

In this example,

$$\alpha = 0.8$$

is selected in order to demonstrate the effect of reducing the Newton step size. A constant damping parameter is used only for illustrative purposes.

At the initial point,

$$f(2) = 2^3 - 2 - 2 = 4,$$

and

$$f'(2) = 3(2)^2 - 1 = 11.$$

The standard Newton step would be

$$-\frac{f(2)}{f'(2)} = -\frac{4}{11} \approx -0.36364.$$

Applying the damping parameter gives

$$x_1 = 2 + 0.8(-0.36364).$$

Thus,

$$x_1 \approx 1.70909.$$

Evaluating the function:

$$f(1.70909) \approx 1.28315.$$

At the second iteration,

$$f'(1.70909) = 3(1.70909)^2 - 1 \approx 7.76296.$$

The Newton step becomes

$$-\frac{1.28315}{7.76296} \approx -0.16529.$$

Applying damping:

$$x_2 = 1.70909 + 0.8(-0.16529).$$

Therefore,

$$x_2 \approx 1.57686.$$

Evaluating the function:

$$f(1.57686) \approx 0.34397.$$

At the third iteration,

$$f'(1.57686) = 3(1.57686)^2 - 1 \approx 6.45946.$$

The Newton step is

$$-\frac{0.34397}{6.45946} \approx -0.05325.$$

Applying damping gives

$$x_3 = 1.57686 + 0.8(-0.05325),$$

which yields

$$x_3 \approx 1.53426.$$

Table 3.4 summarizes the iterations.

Table 3.4: Damped Newton method for $f(x) = x^3 - x - 2$

Iteration	x_k	$f(x_k)$
0	2.00000	4.00000
1	1.70909	1.28315
2	1.57686	0.34397
3	1.53426	0.07730
4	1.52406	0.01594
5	1.52192	0.00321

Compared to the standard Newton method, the damped version produces a more conservative sequence of iterates. The residual decreases more gradually, but the method behaves in a more controlled manner. This illustrates the trade-off between convergence speed and stability that frequently appears in optimization and nonlinear equation solving.

3.6.5 Brent's Method

Brent's method begins with a bracketing interval satisfying

$$f(a)f(b) < 0.$$

For the nonlinear equation

$$f(x) = x^3 - x - 2,$$

the interval

$$[1, 2]$$

is selected because

$$f(1) = -2, \quad f(2) = 4.$$

Therefore, the interval contains at least one root.

Unlike the bisection method, Brent's method does not rely solely on interval halving. Instead, it combines several strategies, including bisection, secant interpolation, and inverse quadratic interpolation. The main objective is to preserve the robustness of bracketing methods while accelerating convergence using interpolation techniques.

At the beginning of the iteration process, the method may behave similarly to the secant method. Using the initial points

$$x_0 = 1, \quad x_1 = 2,$$

the secant approximation gives

$$x_2 = 2 - 4 \frac{2 - 1}{4 - (-2)} = 1.33333.$$

Evaluating the function:

$$f(1.33333) \approx -0.96298.$$

Since the sign change still occurs between

$$1.33333 \quad \text{and} \quad 2,$$

the bracketing condition is preserved.

At later iterations, the method attempts to improve convergence by constructing interpolation models using several previous approximations. For example, an inverse quadratic interpolation step produces the approximation

$$x_3 \approx 1.58280,$$

with

$$f(1.58280) \approx 0.38252.$$

The sign change now occurs between

$$1.33333 \quad \text{and} \quad 1.58280.$$

The interval containing the root is therefore updated while preserving numerical safety. Subsequent interpolation steps generate

$$x_4 \approx 1.51188,$$

with

$$f(1.51188) \approx -0.05605,$$

followed by

$$x_5 \approx 1.52094,$$

where

$$f(1.52094) \approx -0.00261.$$

Finally, the method reaches the approximation

$$x \approx 1.52138,$$

which is very close to the actual root.

Table 3.5 summarizes the iteration sequence.

Brent's method converges rapidly while still maintaining a valid bracketing interval throughout the computation. This combination of robustness and efficiency is one of the main reasons why the method is widely used in practical numerical software libraries.

3.6.6 Comparison for the Example

The numerical results obtained above show clear differences among the methods. Bisection is the slowest method, but it behaves in a very predictable way. Secant converges faster and avoids derivative evaluation. Newton's method gives the fastest convergence in this example, while damped Newton sacrifices some speed in exchange for additional stability. Brent's method combines reliability and efficiency in a balanced manner.

Table 3.5: Brent's method for $f(x) = x^3 - x - 2$

Iteration	Approximation	$f(x_k)$
1	1.33333	-0.96298
2	1.58280	0.38252
3	1.51188	-0.05605
4	1.52094	-0.00261
5	1.52138	0.00000

Table 3.6: General comparison of the methods for the illustrative example

Method	Initial Value(s)	Approximate Root	General Behavior
Bisection	$[1, 2]$	1.52138	Reliable but slow
Secant	$x_0 = 1, x_1 = 2$	1.52138	Faster, derivative-free
Newton	$x_0 = 1.5$	1.52138	Very fast near the root
Damped Newton	$x_0 = 2, \alpha = 0.8$	1.52138	Safer, but slower
Brent	$[1, 2]$	1.52138	Robust and efficient

3.6.7 Failure Cases

Bisection Method. The bisection method requires an initial interval where the function changes sign. Consider

$$f(x) = x^2 + 1.$$

For any real numbers a and b , we have $f(a) > 0$ and $f(b) > 0$. For example, on the interval $[-1, 1]$,

$$f(-1) = 2, \quad f(1) = 2.$$

Since there is no sign change, the method cannot be applied and no iterations can be performed.

Secant Method. The secant method may fail when two consecutive function values are very close, leading to a nearly zero denominator. Consider

$$f(x) = (x - 1)^2,$$

with initial guesses

$$x_0 = 0.9, \quad x_1 = 1.1.$$

Then

$$f(x_0) = 0.01, \quad f(x_1) = 0.01.$$

The update formula

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

contains the denominator $f(x_k) - f(x_{k-1})$, which becomes zero in this case. As a result, the method breaks down and cannot proceed.

Newton's Method. Newton's method may converge slowly or behave poorly when the derivative is small. Consider

$$f(x) = x^3,$$

with initial guess $x_0 = 0.1$. Then

$$f'(x) = 3x^2, \quad f'(0.1) = 0.03.$$

The first iteration gives

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 0.1 - \frac{0.001}{0.03} \approx 0.0667.$$

Subsequent iterations produce only small changes:

$$x_2 \approx 0.0444, \quad x_3 \approx 0.0296.$$

The convergence becomes very slow because the derivative is close to zero near the root.

Damped Newton Method. The damped Newton method may become inefficient if the step size is reduced too aggressively. Consider again

$$f(x) = x^3 - x - 2,$$

with initial guess $x_0 = 2$ and a small damping parameter, for example $\alpha = 0.1$.

The Newton step at x_0 is approximately -0.3636 , so the damped update becomes

$$x_1 = 2 + 0.1 \cdot (-0.3636) \approx 1.9636.$$

The reduction in the function value is limited:

$$f(2) = 4, \quad f(1.9636) \approx 3.600.$$

Compared to the standard Newton method, which moves directly close to the root, the damped version progresses much more slowly. Repeated small steps lead to inefficient convergence.

Brent's Method. Brent's method requires a valid bracketing interval. Consider again

$$f(x) = x^2 + 1.$$

On the interval $[-1, 1]$,

$$f(-1) = 2, \quad f(1) = 2,$$

so there is no sign change. Since Brent's method relies on bracketing, it cannot be initialized in this case. Even though the method is robust, it still depends on the existence of an initial interval containing a root.

3.7 Comparison of Methods

Numerical methods that can be used to solve single nonlinear equations have been described in Section II. In this part, a comparison of their properties is done.

The bisection method stands out because it is the most robust method. Convergence is guaranteed if the initial interval meets the sign condition and the equation is continuous within it. However, its convergence rate is rather low, namely, linear. Hence, the process takes much time [1, 2].

The secant method is more efficient than the former one because the latter does not need any derivations to be performed and the former guarantees quadratic convergence. The secant method has a faster rate than the bisection method: its rate is superlinear. At the same time, convergence is not guaranteed [3].

The fastest algorithm in the list above is the Newton method. This algorithm has a quadratic convergence rate as long as the initial value is close enough to the solution, and the first-order derivation is positive. This property makes it a very fast method of solving equations. Yet, it can diverge depending on the initial value [1, 3].

The damped Newton method differs from the former one only in that it introduces an additional parameter – step size. This modification prevents the method from diverging. The method becomes more robust as it starts converging from a bigger set of values. As far as the convergence rate is concerned, it is reduced because of the new parameter [3, 6].

As was mentioned above, some of the described algorithms are bracketing methods, whereas others are open methods. Brent’s method is the combination of these two types. It is as robust as the bisection method, yet incorporates interpolation techniques used in the secant method [5, 4].

It is worth saying that usually the better the algorithm converges, the less robust it becomes. For example, Brent’s algorithm combines robustness and efficiency, whereas the Newton algorithm becomes unstable if the initial value is taken incorrectly.

4. Systems of Nonlinear Equations

4.1 Multivariable Newton Method

Newton's method can be extended to systems of nonlinear equations in a natural way. Consider a system of the form

$$F(\mathbf{x}) = \mathbf{0}, \quad (4.1)$$

where $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector-valued function.

Starting from an initial guess $\mathbf{x}_0$, the method generates a sequence $\{\mathbf{x}_k\}$ using the iterative formula

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k, \quad (4.2)$$

where the step $\mathbf{s}_k$ is obtained by solving the linear system

$$J_F(\mathbf{x}_k)\mathbf{s}_k = -F(\mathbf{x}_k). \quad (4.3)$$

Here, $J_F(\mathbf{x}_k)$ denotes the Jacobian matrix evaluated at $\mathbf{x}_k$. This formulation can be interpreted as a multivariable generalization of the Newton update, where the nonlinear system is locally approximated by a linear one.

At each iteration, a linear system must be solved. Therefore, compared to the scalar case, the computational cost per iteration is higher. However, under suitable conditions, the method retains its rapid convergence properties.

If the initial guess is sufficiently close to the solution and the Jacobian matrix is nonsingular at the root, the method typically exhibits quadratic convergence. This makes it one of the most efficient methods for solving nonlinear systems [3, 1].

Despite its strong convergence properties, the method has several limitations. The need to compute and factorize the Jacobian matrix can be computationally expensive, especially for large systems. In addition, if the Jacobian is singular or nearly singular, the method may fail or produce unstable steps.

For these reasons, modifications and alternative methods are often considered in practice, particularly when dealing with large-scale or ill-conditioned systems.

Implementation Example. The following implementation shows the core computational structure of Newton's method for nonlinear systems. At each iteration, the Jacobian system is solved in order to compute the Newton step.

```
1 def newton_system(F, J, x0, tol=1e-8, max_iter=100):  
2  
3     x = np.array(x0, dtype=float)  
4     history = []  
5  
6     for i in range(max_iter):
```

```

7      Fx = F(x)
8      norm_Fx = np.linalg.norm(Fx)
9
10
11     if norm_Fx <= tol:
12         return x, history
13
14     Jx = J(x)
15     s = np.linalg.solve(Jx, -Fx)
16
17     x_next = x + s
18     step_norm = np.linalg.norm(s)
19
20     history.append({
21         "iteration": i,
22         "x": x.tolist(),
23         "norm_f": norm_Fx,
24         "step_norm": step_norm
25     })
26
27     if step_norm <= tol:
28         return x_next, history
29
30     x = x_next
31
32     return x, history

```

Listing 4.1: Python implementation of Newton’s method for systems

4.2 Convergence Analysis

The convergence behavior of Newton’s method for systems is similar to the scalar case, but it depends on additional factors related to the structure of the system.

Let $\mathbf{x}^*$ be a solution of the system $F(\mathbf{x}) = \mathbf{0}$. If the function F is sufficiently smooth and the Jacobian matrix $J_F(\mathbf{x}^*)$ is nonsingular, then Newton’s method exhibits quadratic convergence in a neighborhood of $\mathbf{x}^*$. This means that the error satisfies

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \approx C \|\mathbf{x}_k - \mathbf{x}^*\|^2, \quad (4.4)$$

for some constant $C > 0$ and sufficiently large k [3].

This rapid convergence makes Newton’s method very efficient when a good initial approximation is available. However, the method is not globally convergent. If the initial guess is far from the solution, the method may fail to converge or may converge to an unintended solution.

Another important factor is the conditioning of the Jacobian matrix. If $J_F(\mathbf{x}_k)$ is ill-conditioned or nearly singular, solving the linear system

$$J_F(\mathbf{x}_k)\mathbf{s}_k = -F(\mathbf{x}_k) \quad (4.5)$$

may lead to large or unstable steps. This can cause divergence or oscillatory behavior.

In practice, the convergence of the method is often monitored using a combination of criteria such as

$$\|\mathbf{F}(\mathbf{x}_k)\| < \varepsilon \quad (4.6)$$

and

$$\|\mathbf{x}_{k+1} - \mathbf{x}_k\| < \varepsilon, \quad (4.7)$$

where ε is a prescribed tolerance.

Due to these limitations, various modifications of Newton's method have been developed to improve its robustness. These include damping strategies, line search techniques, and quasi-Newton methods such as Broyden's method.

4.3 Broyden's Method

Broyden's method is a quasi-Newton method developed for solving systems of nonlinear equations. The main idea of the method is to avoid the explicit computation of the Jacobian matrix while still retaining a structure similar to Newton's method.

In Newton's method, the step $\mathbf{s}_k$ is obtained by solving

$$J_F(\mathbf{x}_k)\mathbf{s}_k = -F(\mathbf{x}_k). \quad (4.8)$$

In contrast, Broyden's method replaces the Jacobian matrix with an approximation B_k , leading to the system

$$B_k\mathbf{s}_k = -F(\mathbf{x}_k). \quad (4.9)$$

The approximation B_k is updated at each iteration using information from previous steps. One common update formula, known as the good Broyden update, is given by

$$B_{k+1} = B_k + \frac{(\mathbf{y}_k - B_k\mathbf{s}_k)\mathbf{s}_k^T}{\mathbf{s}_k^T\mathbf{s}_k}, \quad (4.10)$$

where

$$\mathbf{y}_k = F(\mathbf{x}_{k+1}) - F(\mathbf{x}_k). \quad (4.11)$$

This update ensures that the new approximation B_{k+1} satisfies a secant-like condition, which is a generalization of the secant method to higher dimensions.

One of the main advantages of Broyden's method is that it eliminates the need to compute the Jacobian matrix explicitly. This can significantly reduce computational cost, especially for large systems or when evaluating derivatives is expensive.

However, the convergence of Broyden's method is generally slower than that of Newton's method. While Newton's method achieves quadratic convergence under suitable conditions, Broyden's method typically exhibits superlinear convergence.

The performance of the method also depends on the choice of the initial approximation B_0 . A good initial approximation can improve convergence speed, while a poor choice may lead to slow convergence or instability.

Despite these limitations, Broyden's method is widely used in practice due to its efficiency and its ability to handle problems where the Jacobian is difficult to compute [3].

Implementation Example. The following implementation fragment illustrates the main structure of Broyden's method. Instead of recomputing the exact Jacobian matrix at every iteration, the method updates an approximation using a rank-one correction formula.

```

1 def broyden(F, x0, B0=None, tol=1e-8, max_iter=100):
2
3     x = np.array(x0, dtype=float)
4
5     if B0 is None:
6         B = np.eye(len(x))
7     else:
8         B = np.array(B0, dtype=float)
9
10    Fx = F(x)
11    history = []
12
13    for i in range(max_iter):
14
15        s = np.linalg.solve(B, -Fx)
16
17        x_next = x + s
18        Fx_next = F(x_next)
19
20        history.append({
21            "iteration": i,
22            "x": x.tolist(),
23            "norm_f": np.linalg.norm(Fx),
24            "step_norm": np.linalg.norm(s)
25        })
26
27        if np.linalg.norm(Fx_next) <= tol:
28            return x_next, history
29
30        y = Fx_next - Fx
31
32        B += np.outer((y - B @ s), s) / (s @ s)
33
34        x = x_next
35        Fx = Fx_next
36
37    return x, history

```

Listing 4.2: Python implementation of Broyden's method

4.4 Illustrative Example

To illustrate the behavior of the methods for systems of nonlinear equations, consider the following system:

$$\begin{cases} x^2 + y^2 - 4 = 0, \\ x - y - 1 = 0. \end{cases} \quad (4.12)$$

This system represents the intersection of a circle and a line. One of its solutions is approximately

$$\mathbf{x}^* \approx (1.82288, 0.82288).$$

The same system is used to demonstrate both Newton's method and Broyden's method. The goal is to observe how the iterates approach the solution.

4.4.1 Newton Method for Systems

Consider the nonlinear system

$$\begin{cases} x^2 + y^2 - 4 = 0, \\ x - y - 1 = 0. \end{cases} \quad (4.13)$$

The initial approximation is selected as

$$\mathbf{x}_0 = (2, 1).$$

This starting point is chosen because it lies relatively close to the expected intersection region of the circle and the line. In Newton-type methods, selecting an initial approximation near the solution generally improves convergence behavior and reduces the risk of divergence.

The vector-valued function is

$$F(x, y) = \begin{bmatrix} x^2 + y^2 - 4 \\ x - y - 1 \end{bmatrix},$$

and the Jacobian matrix is

$$J_F(x, y) = \begin{bmatrix} 2x & 2y \\ 1 & -1 \end{bmatrix}.$$

At the initial point

$$(2, 1),$$

the function value becomes

$$F(2, 1) = \begin{bmatrix} 2^2 + 1^2 - 4 \\ 2 - 1 - 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

The Jacobian matrix evaluated at the initial point is

$$J_F(2, 1) = \begin{bmatrix} 4 & 2 \\ 1 & -1 \end{bmatrix}.$$

Newton's method requires solving the linear system

$$J_F(\mathbf{x}_0)\mathbf{s}_0 = -F(\mathbf{x}_0).$$

Thus,

$$\begin{bmatrix} 4 & 2 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

From the second equation,

$$s_1 - s_2 = 0,$$

so

$$s_1 = s_2.$$

Substituting into the first equation:

$$4s_1 + 2s_1 = -1.$$

Therefore,

$$6s_1 = -1, \quad s_1 = s_2 = -\frac{1}{6}.$$

The Newton step is therefore

$$\mathbf{s}_0 = \left(-\frac{1}{6}, -\frac{1}{6} \right).$$

Updating the approximation:

$$\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{s}_0 = \left(2 - \frac{1}{6}, 1 - \frac{1}{6} \right).$$

Thus,

$$\mathbf{x}_1 = (1.83333, 0.83333).$$

Evaluating the residual norm:

$$\|F(\mathbf{x}_1)\| \approx 0.05556.$$

At the second iteration, the Jacobian matrix becomes

$$J_F(1.83333, 0.83333) = \begin{bmatrix} 3.66666 & 1.66666 \\ 1 & -1 \end{bmatrix}.$$

Solving the corresponding Newton system produces the next approximation

$$\mathbf{x}_2 = (1.82292, 0.82292),$$

with residual norm

$$\|F(\mathbf{x}_2)\| \approx 0.00022.$$

A third iteration gives

$$\mathbf{x}_3 = (1.82288, 0.82288),$$

where the residual becomes extremely small.

Table 4.1 summarizes the iteration sequence.

Table 4.1: Newton method for the nonlinear system

Iteration	x_k	y_k	$\ F(\mathbf{x}_k)\ $
0	2.00000	1.00000	1.00000
1	1.83333	0.83333	0.05556
2	1.82292	0.82292	0.00022
3	1.82288	0.82288	0.00000

The numerical results demonstrate the rapid convergence behavior of Newton's method for nonlinear systems. Once the iteration approaches the solution, the residual norm decreases very quickly, which is consistent with the quadratic convergence properties discussed earlier.

4.4.2 Broyden's Method for Systems

For Broyden's method, the same nonlinear system is considered:

$$\begin{cases} x^2 + y^2 - 4 = 0, \\ x - y - 1 = 0. \end{cases} \quad (4.14)$$

The initial approximation is again selected as

$$\mathbf{x}_0 = (2, 1).$$

As in Newton's method, this point is chosen because it lies relatively close to the expected solution and produces a well-defined Jacobian structure.

Unlike Newton's method, Broyden's method does not recompute the exact Jacobian matrix at every iteration. Instead, the method constructs an approximation that is updated iteratively.

For the initial approximation of the Jacobian matrix, the exact Jacobian evaluated at the initial point is used:

$$B_0 = \begin{bmatrix} 4 & 2 \\ 1 & -1 \end{bmatrix}.$$

This choice provides a stable starting approximation and improves the early convergence behavior of the method.

At the initial iteration, the nonlinear system is

$$F(2, 1) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

The step is obtained by solving

$$B_0 \mathbf{s}_0 = -F(\mathbf{x}_0).$$

Thus,

$$\begin{bmatrix} 4 & 2 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

This produces the same first step as Newton's method:

$$\mathbf{s}_0 = \left(-\frac{1}{6}, -\frac{1}{6} \right).$$

Therefore,

$$\mathbf{x}_1 = (1.83333, 0.83333).$$

The residual norm becomes

$$\|F(\mathbf{x}_1)\| \approx 0.05556.$$

At this stage, instead of recomputing the exact Jacobian, Broyden's method updates the approximation matrix using the vectors

$$\mathbf{s}_0 = \mathbf{x}_1 - \mathbf{x}_0$$

and

$$\mathbf{y}_0 = F(\mathbf{x}_1) - F(\mathbf{x}_0).$$

The update formula is

$$B_1 = B_0 + \frac{(\mathbf{y}_0 - B_0 \mathbf{s}_0) \mathbf{s}_0^T}{\mathbf{s}_0^T \mathbf{s}_0}.$$

Using the updated approximation matrix, the next step is computed and the iteration proceeds.

At the second iteration, the approximation becomes

$$\mathbf{x}_2 = (1.82353, 0.82353),$$

with residual norm

$$\|F(\mathbf{x}_2)\| \approx 0.00346.$$

A third iteration gives

$$\mathbf{x}_3 = (1.82288, 0.82288),$$

where the residual norm becomes very small.

Table 4.2 summarizes the iterations.

Table 4.2: Broyden method for the nonlinear system

Iteration	x_k	y_k	$\ F(\mathbf{x}_k)\ $
0	2.00000	1.00000	1.00000
1	1.83333	0.83333	0.05556
2	1.82353	0.82353	0.00346
3	1.82288	0.82288	0.00001
4	1.82288	0.82288	0.00000

Compared to Newton's method, Broyden's method converges more slowly because it uses approximate Jacobian information. However, the method avoids repeated Jacobian evaluations, which may significantly reduce computational cost in large-scale problems or applications where derivative calculations are expensive.

4.4.3 Comparison for the Example

The results show that Newton's method converges faster than Broyden's method in this example. This is expected, since Newton's method uses exact derivative information, while Broyden's method relies on approximations.

On the other hand, Broyden's method avoids the computation of the Jacobian matrix, which can be advantageous in problems where derivative evaluation is expensive.

4.4.4 Failure Cases

Newton Method. Newton's method may fail when the Jacobian matrix becomes singular or nearly singular. Consider the system

$$\begin{cases} x^2 = 0, \\ y = 0. \end{cases} \quad (4.15)$$

The Jacobian matrix is

$$J_F(x, y) = \begin{bmatrix} 2x & 0 \\ 0 & 1 \end{bmatrix}.$$

At the point $(0, y)$, the Jacobian is singular. Starting from

$$\mathbf{x}_0 = (0, 2),$$

we have

$$F(\mathbf{x}_0) = (0, 2), \quad J_F(\mathbf{x}_0) = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

The Newton step requires solving

$$J_F(\mathbf{x}_0)\mathbf{s}_0 = -F(\mathbf{x}_0),$$

which becomes

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{s}_0 = \begin{bmatrix} 0 \\ -2 \end{bmatrix}.$$

This system has no unique solution, since the Jacobian matrix is singular. Therefore, the method cannot proceed from this initial point.

Even when the Jacobian is not exactly singular but close to singular, the computed step may become very large, leading to instability and divergence.

Broyden Method. Broyden's method may fail when the update of the Jacobian approximation becomes numerically unstable. Consider again the system

$$\begin{cases} x^2 + y^2 - 4 = 0, \\ x - y - 1 = 0, \end{cases} \quad (4.16)$$

and assume that two consecutive iterates are very close:

$$\mathbf{x}_k = (1.82288, 0.82288), \quad \mathbf{x}_{k+1} = (1.82289, 0.82289).$$

Then the step

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$$

is extremely small. In the Broyden update formula

$$B_{k+1} = B_k + \frac{(\mathbf{y}_k - B_k \mathbf{s}_k) \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{s}_k},$$

the denominator $\mathbf{s}_k^T \mathbf{s}_k$ becomes very small.

As a result, the update term may become very large, causing a significant distortion in the Jacobian approximation. This can lead to unstable iterations and loss of convergence.

In practice, such situations are handled by introducing safeguards, such as skipping the update or restarting the method with a new approximation.

4.5 Comparison of Methods

There are trade-offs between these two methods.

Newton's method is usually more accurate and converges more quickly provided that the Jacobian matrix is well-conditioned and the starting point is close enough to the root. The requirement to calculate the Jacobian and solve systems using the Jacobian matrix is computationally expensive.

In Broyden's method, on the contrary, the computational overhead is avoided by not calculating the Jacobian explicitly.

In general, Newton's method is better than Broyden's when higher accuracy and speed of convergence are essential. Conversely, Broyden's method is preferred when computational efficiency is crucial [3, 5].

5. Implementation and Numerical Experiments

5.1 Implementation Details

In order to examine numerical methods mentioned above from the point of view of their practical convergence properties and computation features, the corresponding algorithms have been implemented using the Python programming language. Two groups of implementations have been made: one for single nonlinear equations and another for systems of nonlinear equations. During the development of these algorithms, special attention was paid to the consistency, numerical stability, and clearness of the code.

For single nonlinear equations, algorithms of bisection, secant, Newton, damped Newton, and Brent methods were implemented. For nonlinear systems, multivariable Newton and Broyden algorithms were implemented. Every algorithm is built using a unified pattern in order to make comparisons easier and create a unified experimental setting.

Every solver function returns a dictionary which contains the following items: root value, convergence indicator, number of iterations performed, residual vector information, reason for termination of calculations, and iteration history. Using the unified output structure allows comparing the results of several algorithms easily and facilitates creation of comparison tables and graphs.

Criteria of convergence were chosen to be the tolerance based and maximum iteration criteria. For each algorithm, there are certain convergence criteria depending on the method of calculation (residuals norm, intervals width, and size of steps). Information about iterations is stored during the algorithm execution in order to conduct deeper convergence analysis later.

Tests for all functions have been written using the `pytest` module. There were such tests as verification of convergence, test with invalid initial parameters, singularity of Jacobians matrix, maximum iterations criterion, and verification of iteration history. These tests were effective in error finding.

The numerical experiments and graphical comparisons were conducted using the Jupyter notebook environment and NumPy, Pandas, and Matplotlib modules.

5.1.1 Code Organization and Reproducibility

All the numerical methods introduced in this thesis have been programmed in Python, with code written in a consistent and reproducible manner. In order to increase clarity and consistency, different implementation files have been developed for each numerical method.

The repository was organized into multiple components. A simplified version of the project structure is in below.

```
project/  
  src/  
    single_variable/  
      bisection.py  
      secant.py  
      newton.py  
      damped_newton.py  
      brent.py  
  
    systems/  
      newton_system.py  
      broyden.py  
  
    utils/  
  
  notebooks/  
    single_variable_experiments.ipynb  
    system_experiments.ipynb  
  
  tests/  
  figures/  
  requirements.txt
```

The `src/` directory includes the implementations of the solvers that were applied throughout the work. Single variable solvers and systems' solvers were split into separate modules for convenience and reuse.

Numerical experiments, convergence plots, and analyses of iteration histories were carried out with the use of Jupyter notebooks that are saved in the `notebooks/` directory. Scientific Python libraries such as NumPy, Matplotlib, and Pandas were employed in these experiments.

A uniform output format was followed for all solvers. The implementations provide such information about the solving process as approximate solution, convergence success, number of iterations, residuals, and iteration history. The unified output helped in comparing various numerical methods and in producing the results experimentally.

Moreover, automatic testing mechanisms were applied in the process of development for checking the accuracy of implementations. It made possible for the results from Python programs and the theoretical formulations from the work to agree on each other.

The structure of the repository helped in improving reproducibility. Since all experiments were produced by using implemented solvers and Jupyter notebooks, one may easily reproduce and extend the numerical results and visualizations included in the thesis.

Concluding, the practical experience in the process of implementing numerical solvers showed some practical side of the subject.

5.2 Test Problems

Several benchmark problems were selected in order to evaluate the convergence behavior, robustness, and computational efficiency of the implemented numerical methods. Separate test problems were used for single-variable equations and nonlinear systems.

For single-variable experiments, the equation

$$f(x) = \cos(x) - x$$

was considered. This equation was selected because it is a nonlinear transcendental equation whose root cannot be expressed in a simple closed form. In addition, the problem allows the convergence characteristics of different iterative methods to be observed clearly.

The derivative

$$f'(x) = -\sin(x) - 1$$

was used for Newton-based methods.

The initial values and intervals were selected according to the requirements and stability properties of the algorithms. For the bisection and Brent methods, the interval

$$[0, 1]$$

was chosen because the function changes sign over this interval:

$$f(0) = 1 > 0, \quad f(1) = \cos(1) - 1 < 0.$$

Therefore, the interval guarantees the existence of a root according to the Intermediate Value Theorem and provides a valid starting point for bracketing methods.

For Newton's method and the damped Newton method, the initial approximation

$$x_0 = 0.5$$

was selected because it is reasonably close to the root and lies inside the bracketing interval. Choosing an initial point near the root generally improves convergence behavior and reduces the risk of divergence in Newton-type methods.

For the secant method, the initial approximations

$$x_0 = 0, \quad x_1 = 1$$

were selected in order to provide two distinct starting points surrounding the root. Since the secant method approximates derivative information using previous iterates, the choice of initial points directly influences the stability and convergence speed of the algorithm.

For nonlinear system experiments, the following system was used:

$$\begin{cases} x^2 + y^2 - 5 = 0, \\ e^x + y - 1 = 0. \end{cases}$$

This system was selected because it combines nonlinear algebraic and exponential terms while remaining sufficiently small for detailed analysis. The problem is suitable for evaluating the behavior of Jacobian-based methods and quasi-Newton approaches.

The Jacobian matrix of the system is given by

$$J_F(x, y) = \begin{bmatrix} 2x & 2y \\ e^x & 1 \end{bmatrix}.$$

The initial approximation for the nonlinear system experiments was chosen as

$$\mathbf{x}_0 = (1, -1).$$

This initial point was selected because it is relatively close to the intersection region of the nonlinear equations and produces a nonsingular Jacobian matrix. In Newton-type methods for systems, selecting an initial approximation that yields a well-conditioned Jacobian matrix is important for stable convergence behavior.

The selected benchmark problems were intentionally moderate in size and complexity in order to focus primarily on convergence behavior, residual reduction, numerical stability, and method comparison.

5.3 Results and Comparisons

The implemented methods were tested on the selected benchmark problems in order to compare their convergence behavior and computational characteristics. The experiments focused primarily on iteration counts, residual reduction, and stability of the numerical methods.

5.3.1 Single-Variable Experiments

For the scalar nonlinear equation

$$f(x) = \cos(x) - x,$$

all single-variable methods successfully converged to the same root when appropriate initial values were selected. However, the convergence speeds of the methods differed significantly.

Figure 5.1 presents the residual reduction behavior of the implemented methods during the iterative process.

The Newton method converged rapidly and achieved very small residual values within only a few iterations. This behavior is consistent with the quadratic convergence properties discussed in previous chapters.

The secant method also demonstrated fast convergence while avoiding explicit derivative evaluations. Although its convergence was slightly slower than Newton's method, the method remained computationally efficient.

The damped Newton method produced convergence behavior similar to the standard Newton method in this experiment because the selected initial approximation was already sufficiently close to the root. In more difficult problems, damping strategies may provide additional stability.

The bisection method converged more slowly than the other approaches due to its linear convergence behavior. Nevertheless, the method consistently reduced the bracketing interval and remained highly robust throughout the experiment.

Brent's method demonstrated balanced behavior by combining the robustness of bracketing methods with interpolation-based acceleration techniques. Its convergence performance was competitive while maintaining stable numerical behavior.

Table 5.1 summarizes the numerical results obtained for the implemented single-variable methods.

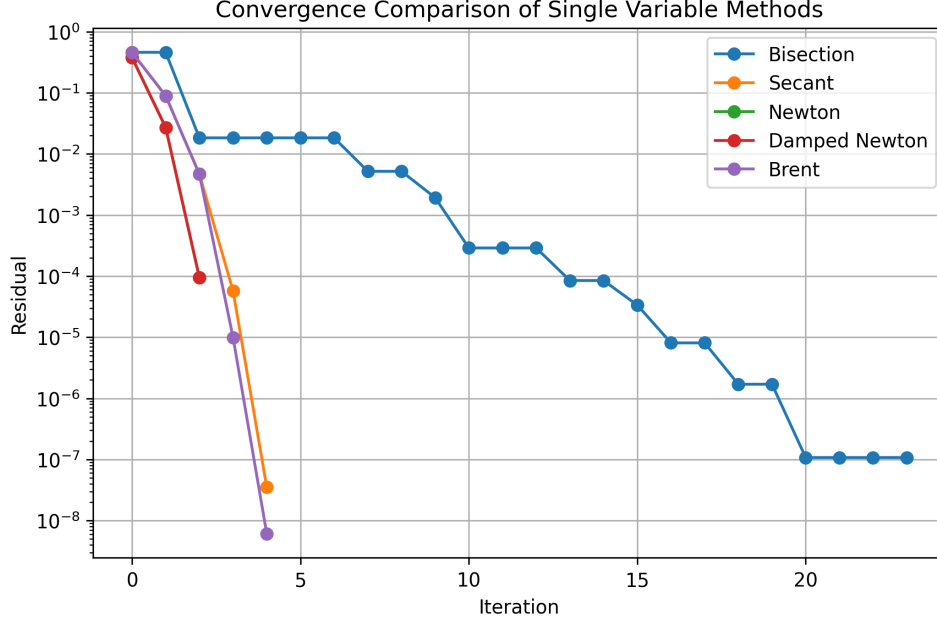


Figure 5.1: Convergence comparison of single-variable methods

Table 5.1: Comparison of single-variable methods

Method	Iterations	Final Residual	Converged
Bisection	24	7.75×10^{-9}	Yes
Secant	5	2.67×10^{-13}	Yes
Newton	3	1.18×10^{-9}	Yes
Damped Newton	3	1.18×10^{-9}	Yes
Brent	5	6.11×10^{-9}	Yes

5.3.2 System Experiments

In addition to single-variable experiments, nonlinear system solvers were tested on a benchmark problem involving coupled nonlinear equations. The experiments focused on comparing the convergence behavior of the multivariable Newton method and Broyden's method.

The following nonlinear system was used:

$$\begin{cases} x^2 + y^2 - 5 = 0, \\ e^x + y - 1 = 0. \end{cases}$$

The initial approximation was selected as

$$\mathbf{x}_0 = (1, -1).$$

Both methods successfully converged to the same solution. However, their convergence characteristics differed due to the use of exact and approximate Jacobian information.

Figure 5.2 presents the residual norm reduction during the iterative process.

The multivariable Newton method converged more rapidly and achieved smaller residual values within fewer iterations. This behavior is consistent with the quadratic convergence properties of Newton-based methods when accurate Jacobian information is available.

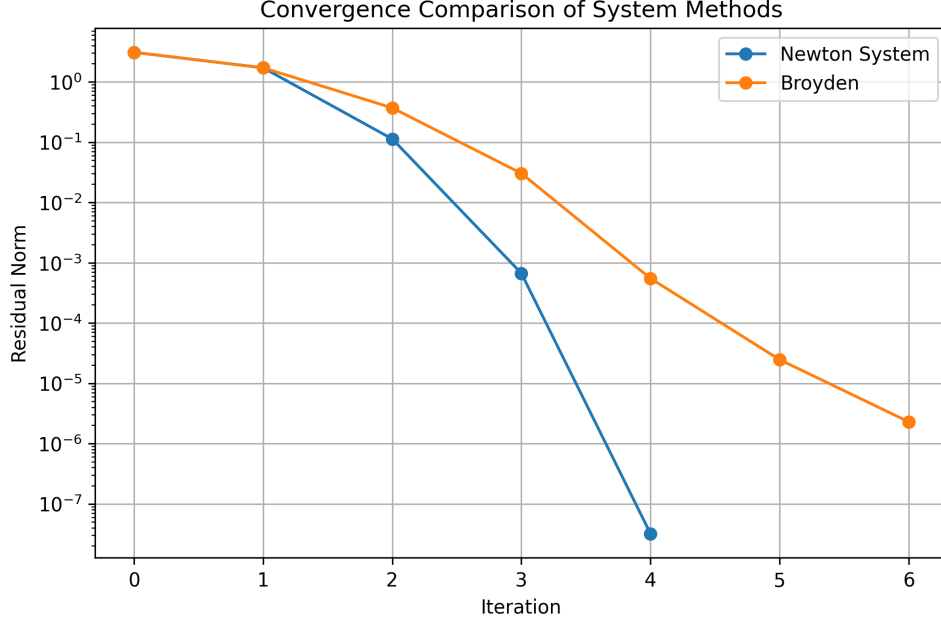


Figure 5.2: Convergence comparison of system methods

Broyden’s method required more iterations because the Jacobian matrix was approximated iteratively instead of being computed exactly at every step. Nevertheless, the method still demonstrated stable convergence behavior and reduced the computational burden associated with repeated Jacobian evaluations.

Table 5.2 summarizes the numerical results obtained for the nonlinear system experiments.

Table 5.2: Comparison of system methods

Method	Iterations	Final Residual	Converged
Newton System	5	2.22×10^{-16}	Yes
Broyden	7	5.35×10^{-10}	Yes

The experimental results show that the theoretical characteristics of the methods are reflected in their computational behavior. Newton-type methods converge rapidly when derivative information is available, while derivative-free or approximate-Jacobian approaches require more iterations but may offer practical advantages in terms of implementation cost.

5.4 Discussion

From the conducted numerical experiments, we can state that there is a variety of advantages and disadvantages of different approaches to solving nonlinear equations depending on the nature of the problem and the information at hand.

The fastest convergence was obtained with the help of the Newton method if good enough initial guesses were used. With the quadratic convergence rate, this method is capable of driving the residual down in a relatively small number of iterations. Nevertheless, the method requires information about the first-order derivative and may lead to

instability.

In cases when derivatives are not easily computable, the secant method becomes quite an alternative to the previously discussed techniques since it does not need derivative information. Still, it is worth noting that in case of poor initial guess, the method could fail in finding solutions.

Another approach that sacrifices the convergence speed but compensates it with the increased stability is the damped Newton's method that manages to control the size of the iterative step thus providing better stability and smoother convergence process.

Finally, the most consistent technique in terms of guaranteed convergence is the bisection method, which is able to find a solution in almost all cases. Though, in the course of the experiment, it became evident that this method is rather slow as compared to others.

Moreover, Brent's method combines both properties, robustness and fast convergence. Thus, it can be said that this is also a very promising approach.

In the context of solving nonlinear systems of equations, the multi-variable Newton method provides fast convergence with good Jacobian approximations. On the other hand, the Broyden method converges a bit slower but decreases the computational complexity.

6. Connection with Optimization

6.1 Residual Minimization Perspective

Nonlinear equation solving and optimization are closely related areas in numerical mathematics. Although the primary objective of root-finding methods is to solve equations of the form

$$f(x) = 0$$

or

$$F(\mathbf{x}) = \mathbf{0},$$

many iterative algorithms can also be interpreted from an optimization perspective.

For a scalar nonlinear equation, solving

$$f(x) = 0$$

is equivalent to minimizing the residual function

$$\phi(x) = |f(x)|.$$

Since absolute values are not always convenient for analysis, a squared formulation is often preferred:

$$\phi(x) = \frac{1}{2}f(x)^2.$$

The factor $\frac{1}{2}$ is introduced mainly for notational simplicity when derivatives are computed.

If x^* is an exact root of the equation, then

$$f(x^*) = 0,$$

and therefore

$$\phi(x^*) = 0.$$

Thus, every root of the equation corresponds to a global minimizer of the residual function.

Similarly, for nonlinear systems

$$F(\mathbf{x}) = \mathbf{0},$$

a residual minimization formulation can be constructed using vector norms:

$$\phi(\mathbf{x}) = \frac{1}{2}\|F(\mathbf{x})\|^2.$$

Using the Euclidean norm,

$$\|F(\mathbf{x})\|^2 = F(\mathbf{x})^T F(\mathbf{x}),$$

the objective function becomes

$$\phi(\mathbf{x}) = \frac{1}{2} F(\mathbf{x})^T F(\mathbf{x}).$$

This formulation transforms the root-finding problem into an unconstrained optimization problem.

The optimization interpretation is particularly useful when exact roots are difficult to compute or when the system is inconsistent. In such cases, instead of finding an exact zero residual solution, numerical methods attempt to minimize the residual norm as much as possible.

However, it is important to note that the converse statement is not always true. A minimizer of

$$\phi(\mathbf{x})$$

does not necessarily correspond to an exact root of the system. In some cases, the residual norm may attain a local minimum while remaining strictly positive.

Therefore, root-finding and optimization problems are closely connected but not completely identical.

6.2 Newton's Method and Optimization

Newton's method can also be interpreted from an optimization perspective. In optimization theory, Newton-type methods are commonly used to minimize objective functions by using second-order derivative information.

Consider the scalar optimization problem

$$\min \phi(x).$$

A stationary point of the function satisfies

$$\phi'(x) = 0.$$

Applying Newton's method to this equation gives

$$x_{k+1} = x_k - \frac{\phi'(x_k)}{\phi''(x_k)}.$$

This expression is known as Newton's optimization method.

Now consider the residual minimization formulation

$$\phi(x) = \frac{1}{2} f(x)^2.$$

The first derivative is

$$\phi'(x) = f(x)f'(x),$$

and the second derivative is

$$\phi''(x) = [f'(x)]^2 + f(x)f''(x).$$

Substituting these expressions into Newton's optimization formula yields

$$x_{k+1} = x_k - \frac{f(x_k)f'(x_k)}{[f'(x_k)]^2 + f(x_k)f''(x_k)}.$$

Near the root, the residual term

$$f(x_k)$$

becomes very small. Therefore, the term

$$f(x_k)f''(x_k)$$

can often be neglected locally. This approximation gives

$$x_{k+1} \approx x_k - \frac{f(x_k)}{f'(x_k)},$$

which is precisely the classical Newton iteration for root finding.

This observation shows that Newton's root-finding method can be interpreted as an approximate optimization method applied to the residual function.

For nonlinear systems, a similar interpretation can be obtained using the objective function

$$\phi(\mathbf{x}) = \frac{1}{2}F(\mathbf{x})^T F(\mathbf{x}).$$

The gradient of this function is given by

$$\nabla \phi(\mathbf{x}) = J_F(\mathbf{x})^T F(\mathbf{x}),$$

where

$$J_F(\mathbf{x})$$

denotes the Jacobian matrix of the system.

Thus, Newton-type methods for nonlinear systems are also closely related to optimization techniques based on gradient and curvature information.

6.3 Damped Newton Method and Line Search

One of the main difficulties of Newton's method is its sensitivity to the initial approximation. Although the method converges rapidly near the solution, large iteration steps may cause instability or divergence when the initial point is far from the root.

The damped Newton method attempts to improve stability by modifying the iteration formula:

$$x_{k+1} = x_k + \alpha_k s_k,$$

where

$$s_k = -\frac{f(x_k)}{f'(x_k)}$$

is the classical Newton step and

$$0 < \alpha_k \leq 1$$

is a damping parameter.

When

$$\alpha_k = 1,$$

the method reduces to the standard Newton iteration. Smaller values of

$$\alpha_k$$

reduce the step size and may prevent excessively large updates.

This idea is closely related to line-search methods in optimization. In optimization algorithms, the search direction is first determined and then an appropriate step length is selected in order to improve stability and guarantee sufficient decrease of the objective function [6].

For residual minimization problems, the objective function can be written as

$$\phi(x) = \frac{1}{2}f(x)^2.$$

Instead of taking the full Newton step immediately, the damped Newton method searches for a step size satisfying

$$\phi(x_k + \alpha_k s_k) < \phi(x_k).$$

Therefore, the algorithm attempts to reduce the residual function at every iteration. A similar idea can also be applied to nonlinear systems:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{s}_k,$$

where

$$\mathbf{s}_k$$

is obtained from the Newton system

$$J_F(\mathbf{x}_k)\mathbf{s}_k = -F(\mathbf{x}_k).$$

The damping strategy is particularly useful when the Jacobian matrix changes rapidly or when the initial approximation is not sufficiently close to the solution.

Although damping may slow down convergence near the solution, it often improves global stability and reduces the risk of divergence. This trade-off between convergence speed and stability is one of the central ideas connecting nonlinear equation solving and optimization methods.

6.4 Trust-Region Perspective

Another important connection between nonlinear equation solving and optimization arises from trust-region methods. These methods are designed to improve the stability of Newton-type algorithms when the local model becomes unreliable [6].

Newton's method is based on a local linear or quadratic approximation of the function near the current iterate. For example, in the scalar case, the method uses the first-order Taylor expansion

$$f(x_k + s) \approx f(x_k) + f'(x_k)s.$$

The Newton step is obtained by forcing this approximation to become zero:

$$f(x_k) + f'(x_k)s_k = 0.$$

This gives

$$s_k = -\frac{f(x_k)}{f'(x_k)}.$$

However, this approximation is only reliable locally. If the current iterate is far from the solution or if the function changes rapidly, the Newton step may become excessively large and lead to instability.

Trust-region methods address this problem by restricting the step size. Instead of accepting arbitrarily large updates, the method searches for a step satisfying

$$\|s_k\| \leq \Delta_k,$$

where

$$\Delta_k > 0$$

is called the trust-region radius.

The basic idea is that the local approximation is trusted only inside a limited neighborhood around the current iterate. If the model produces good predictions, the trust region may be enlarged. Otherwise, the region is reduced in order to obtain more stable steps.

For nonlinear systems, the local linear model is given by

$$F(\mathbf{x}_k + \mathbf{s}) \approx F(\mathbf{x}_k) + J_F(\mathbf{x}_k)\mathbf{s}.$$

Instead of solving the Newton system without restrictions, trust-region methods search for a step satisfying

$$\|\mathbf{s}\| \leq \Delta_k.$$

This restriction prevents excessively large updates and improves robustness, especially when the Jacobian matrix is poorly conditioned or when the initial approximation is far from the solution.

Trust-region ideas are also closely related to damped Newton methods. Both approaches attempt to control the iteration step in order to improve global convergence behavior. While line-search methods reduce the step length after computing a direction, trust-region methods restrict the admissible region before the step is accepted.

In practical optimization algorithms, trust-region techniques are widely used because of their strong stability properties [6]. Similar ideas also appear in modern nonlinear equation solvers, particularly for large-scale and ill-conditioned systems.

6.5 Numerical Illustration of Damping Effects

In order to illustrate the relationship between damping strategies and optimization-based stabilization techniques, an additional numerical experiment was performed using the nonlinear equation

$$f(x) = x^3 - 2x + 2.$$

This function was selected because the classical Newton iteration may exhibit unstable behavior for certain initial approximations. In particular, the method may oscillate between distant points instead of converging to a root.

The derivative of the function is

$$f'(x) = 3x^2 - 2.$$

The initial approximation was chosen as

$$x_0 = 0.$$

This starting point is intentionally difficult for the standard Newton method. The purpose of the experiment is to observe how damping strategies may improve stability when the full Newton step becomes problematic.

Using the standard Newton iteration,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

the iterates behave as follows:

$$x_0 = 0, \quad x_1 = 1, \quad x_2 = 0, \quad x_3 = 1.$$

Thus, the method oscillates between two points and fails to converge.

Now consider the damped Newton iteration

$$x_{k+1} = x_k + \alpha_k s_k,$$

where

$$s_k = -\frac{f(x_k)}{f'(x_k)}$$

and

$$0 < \alpha_k < 1.$$

By reducing the step size, the iteration becomes more stable and avoids the oscillatory behavior observed in the standard Newton method.

This example demonstrates one of the main motivations behind damping and line-search techniques. Large Newton steps may be efficient near the solution, but they can also produce unstable behavior when the local approximation is unreliable. Controlling the step size improves robustness and is closely related to optimization-based globalization strategies.

7. Conclusion

7.1 Summary of Findings

As a part of the research described in this thesis, several numerical techniques used for finding roots of nonlinear equations or systems of nonlinear equations is considered.

For case of a single nonlinear equation, the following techniques is reviewed: bisection, secant, Newton, damped Newton, and Brent algorithms. Their formulas, convergences, advantages and disadvantages, and theoretical and experimental evidence have been provided.

The methods considered for nonlinear systems include multivariable Newton and Broyden algorithms. The concepts of Jacobian matrix, modes of convergence, and quasi-Newton approaches have been presented both in theory and via examples.

Furthermore, Python implementations of all methods considered in this thesis have been created. While implementing, issues of consistency and reproducibility of experiments have been taken into account. The obtained results included visualizing convergence curves, comparison tables, and residual analysis with the help of Jupyter notebooks.

At the same time, the connection of nonlinear equation solving with optimization problems has been highlighted in the paper. Residual minimization formulations, damping, line search, and trust region viewpoints have been introduced to show the connection of root finding algorithms with optimization techniques.

As can be seen from results, nonlinear equations are not just mathematical theoretical problems. They are essential components of scientific computing.

7.2 Limitations

The range of applicability of this dissertation was consciously restricted to nonlinear problems of relatively small to medium size. Sparse large-scale systems of equations, optimization techniques for high-dimensional problems, and sophisticated methods of global search were not covered in this work.

Moreover, the experimentation was concentrated mostly on the investigation of convergence properties and rate of reduction of the residual vector, rather than computing time or large scale performance.

Finally, some more sophisticated approaches, including adaptive trust region schemes, Levenberg-Marquardt method, and quasi-Newton techniques of second order, have not been studied in this work.

7.3 Future Work

Several possible extensions of this work can be considered in future studies.

Some possible directions of development of this work are discussed below.

The first possible area of research is the analysis of very large scale non-linear systems that emerge in scientific computation and engineering fields. In this case, efficient use of sparsity, iterative linear algebra solvers, and other similar approaches becomes more and more important.

Secondly, one can investigate some advanced numerical non-linear solvers based on optimization. One can apply line-search and trust region methods as well as quasi-Newton approaches.

Thirdly, existing implementations can also be integrated into an overall numerical software package that includes some visualization features, benchmarking capabilities, and automatic tests.

Lastly, the intersection between non-linear equations solution, optimization, and machine learning algorithms for optimization may become a topic for future research.

Bibliography

- [1] Burden, R. L., & Faires, J. D. (2011). *Numerical analysis* (9th ed.). Brooks/Cole.
- [2] Atkinson, K. (1989). *An introduction to numerical analysis* (2nd ed.). John Wiley & Sons.
- [3] Kelley, C. T. (2003). *Solving nonlinear equations with Newton's method*. SIAM.
- [4] University of Illinois at Urbana-Champaign. (n.d.). *Solving nonlinear equations*. Retrieved from https://cs357.cs.illinois.edu/textbook/notes/solve_nd.html
- [5] Chapra, S. C., & Canale, R. P. (2015). *Numerical methods for engineers* (7th ed.). McGraw-Hill.
- [6] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd Edition, Springer, 2006.

A. Resume

Name Surname	Serhet Gökdemir
Birth Date	19.11.2001
Birth Place	Diyarbakır
High School	2016–2020 Diyarbakır Anadolu Lisesi
Internships	Turkcell Teknoloji A.Ş Ziraat Teknoloji A.Ş (Ongoing)